EXAMEN PRATIQUE — APPLICATION FULL STACK MERN

Node.js • Express • MongoDB Atlas • Mongoose • HTML/CSS/JavaScript • Git/GitHub • Vercel • Railway

Nom et prénom : ..... **Date :** .....

Durée indicative : 3 h 30 **Travail :** individuel **Livrables :** dépôt GitHub, URL de l'application, démonstration

Objectif : créer et déployer une petite application MERN fonctionnelle, versionnée et configurée avec des variables d'environnement.

RÈGLE ABSOLUE DES COMMITS

Chaque question et chaque étape demandée doit être validée par un commit Git identifiable. **Une question/étape sans commit correspondant vaut 0 pour cette question/étape.** Les commits doivent être réalisés au fur et à mesure : un seul commit final ne remplace pas l'historique demandé.

Sécurité — consignes obligatoires

• Ne jamais committer un fichier .env, un mot de passe, une URI MongoDB contenant des identifiants, une clé secrète ou un jeton. Ajouter les fichiers sensibles à .gitignore.

• Pour l'exercice, la règle MongoDB Atlas 0.0.0.0/0 est demandée afin d'autoriser les connexions depuis toutes les adresses IP. Elle expose davantage la base : utiliser uniquement une base de test, un utilisateur à privilèges minimaux, puis retirer cette règle après l'évaluation.

• Créer un utilisateur de base de données dont le nom est le prénom de l'étudiant. Le mot de passe doit suivre le modèle demandé par l'enseignant : prénom + « 12345 » + date de naissance. Ne pas publier ni committer ce mot de passe ; le transmettre uniquement dans les paramètres secrets de l'environnement.

• Utiliser des secrets fictifs pour toute capture d'écran ou démonstration publique. Ne pas exposer de données personnelles réelles.

Partie 1 — Initialisation Git et structure du projet (10 points)

Question 1.1 — Créer le dépôt local (4 pts)

Créez un dossier de projet nommé **mern-prenom-nom**, initialisez Git et créez un dépôt GitHub public ou privé selon les consignes de l'enseignant. Reliez le dépôt local au dépôt distant.

```
mkdir mern-prenom-nom
cd mern-prenom-nom
git init
mkdir backend frontend
git add .
git commit -m "chore: initialiser le projet MERN"
git branch -M main
git remote add origin <URL_DU_DEPOT_GITHUB>
git push -u origin main
```

Livrable : dépôt GitHub accessible à l'enseignant et premier commit visible.

Question 1.2 — Arborescence (3 pts)

Le dépôt doit contenir au minimum les deux dossiers suivants :

```
mern-prenom-nom/
■■■ backend/
■ ■■■ index.js
■ ■■■ package.json
■ ■■■ .env.example
■■■ frontend/
■ ■■■ index.html
```

```
■ ■■■ style.css
■ ■■■ app.js
■■■ .gitignore
■■■ README.md
```

Le fichier .env.example contient uniquement des noms de variables et des valeurs fictives, jamais de vrais secrets.

Question 1.3 — Commits (3 pts)

Effectuez un commit distinct à la fin de cette question. Exemple :

```
git add .
git commit -m "chore: creer la structure frontend et backend"
```

Partie 2 — Backend Node.js et Express (15 points)

Question 2.1 — Initialisation et dépendances (4 pts)

Dans backend/, initialisez npm et installez express, mongoose, dotenv et cors. Installez nodemon comme dépendance de développement.

```
cd backend
npm init -y
npm install express mongoose dotenv cors
npm install --save-dev nodemon
```

Question 2.2 — Scripts npm (4 pts)

Configurez package.json avec un démarrage de production et un script de développement personnalisé. Remplacez **prenom** par votre prénom, en minuscules et sans espace.

```
"scripts": {
"start": "node index.js",
"prenom": "nodemon index.js"
}
```

Exemple de lancement : **npm start** pour démarrer avec Node.js et **npm run othman** si le prénom de l'étudiant est Othman. Le script personnalisé doit être nommé d'après le prénom réel de l'étudiant.

Question 2.3 — Serveur et routes (4 pts)

Créez index.js. Le serveur doit charger les variables d'environnement, écouter sur process.env.PORT ou le port 3000, et exposer :

• GET /api/health : renvoie un JSON confirmant que le serveur répond.

• GET /api/items : renvoie une liste JSON d'éléments de démonstration ou les éléments lus en base.

• POST /api/items : valide les champs reçus et crée un élément en base.

Gérez les erreurs avec des codes HTTP appropriés et activez express.json().

Question 2.4 — Commit (3 pts)

```
git add backend
git commit -m "feat: creer le backend Express"
```

Partie 3 — MongoDB Atlas et Mongoose (15 points)

Question 3.1 — Base de données (5 pts)

Créez un cluster de test MongoDB Atlas, une base de données et un utilisateur dédié. Le nom d'utilisateur suit la consigne de l'enseignant. Attribuez uniquement les droits nécessaires à la base de test.

Dans Network Access, ajoutez **0.0.0.0/0** pour cet exercice uniquement. Cette règle signifie « toutes les adresses IPv4 » ; elle ne doit pas être utilisée pour une base de production exposée.

Question 3.2 — Connexion Mongoose (5 pts)

Placez l'URI de connexion dans la variable MONGODB_URI du fichier backend/.env. Chargez dotenv et connectez Mongoose avant de considérer l'API prête. Créez un schéma et un modèle Mongoose pour les éléments de l'application.

```
MONGODB_URI=mongodb+srv://<utilisateur>:<mot_de_passe>@<cluster>/<base>?retryWrites=true&w=majority
PORT=3000
```

Cette URI est un exemple de format seulement. Remplacez les champs par les valeurs de votre cluster et n'ajoutez jamais le vrai fichier .env à Git.

Question 3.3 — Vérification et commit (5 pts)

Vérifiez que le serveur se connecte à Atlas et que les routes lisent/écrivent dans la base. Ajoutez un fichier .env.example avec des valeurs factices, puis commitez le code sans secrets.

```
git add backend/index.js backend/package.json backend/.env.example .gitignore
git commit -m "feat: connecter MongoDB avec Mongoose"
```

Partie 4 — Frontend HTML, CSS et JavaScript (10 points)

Question 4.1 — Interface (5 pts)

Dans frontend/, réalisez une interface responsive en HTML/CSS/JavaScript natif. L'utilisateur doit pouvoir afficher les éléments, ajouter un élément via un formulaire et voir les erreurs de requête.

Question 4.2 — Appels API (3 pts)

Le frontend doit appeler l'API via fetch(). N'inscrivez pas une URL de backend en dur dans plusieurs fichiers : centralisez-la dans une configuration de base. Pour un site statique, la configuration doit être fournie au déploiement ou générée dans un fichier de configuration non secret.

Question 4.3 — Commit (2 pts)

```
git add frontend
git commit -m "feat: creer l interface HTML CSS JS"
```

Partie 5 — GitHub, documentation et hygiène Git (10 points)

• Poussez chaque commit sur GitHub : git push.

• Vérifiez que l'historique est lisible, avec un commit par question/étape.

• Complétez README.md : objectif, prérequis, installation, commandes, variables requises, routes API et liens de déploiement.

• Vérifiez que .gitignore exclut au minimum node_modules/, .env, .env.* (tout en autorisant .env.example) et les fichiers de build pertinents.

```
git status
git log --oneline --decorate
git push
```

Commit demandé : **docs: documenter installation et deploiement**

Partie 6 — Déploiement frontend sur Vercel (10 points)

• Importez le dépôt GitHub dans Vercel.

• Configurez le dossier racine du projet sur frontend/ et déployez l'interface statique.

• Vérifiez que le site est accessible via l'URL Vercel et qu'il peut appeler l'API.

• Si nécessaire, configurez les règles CORS du backend pour autoriser uniquement les origines frontend prévues.

Fournissez l'URL Vercel et faites un commit de documentation après validation.

```
git add README.md
git commit -m "deploy: documenter le frontend Vercel"
```

```
git push
```

Partie 7 — Deux backends : Vercel et Railway (15 points)

Question 7.1 — Backend Vercel (5 pts)

Déployez le dossier backend sur Vercel comme fonction/API Node.js. Ajoutez une configuration compatible avec le routage des fonctions et vérifiez GET /api/health. Le backend Vercel doit disposer de ses propres variables d'environnement.

Question 7.2 — Backend Railway (5 pts)

Déployez le même code backend sur Railway. Configurez les variables d'environnement dans le tableau de bord Railway, vérifiez les logs et testez GET /api/health. Ne supposez pas que la configuration Vercel est automatiquement partagée avec Railway.

Question 7.3 — Variables de production (3 pts)

Configurez séparément sur Vercel et Railway les variables nécessaires, notamment MONGODB_URI, NODE_ENV et toute autre variable utilisée par le projet. Ne commitez aucun secret. Le PORT est généralement fourni par la plateforme ; le code doit utiliser process.env.PORT avec une valeur de repli en local.

Question 7.4 — Commit (2 pts)

```
git add .
git commit -m "deploy: configurer les backends Vercel et Railway"
git push
```

Partie 8 — Bascule automatique entre les deux backends (10 points)

Le frontend doit essayer le backend principal, puis utiliser l'autre backend si le premier échoue. L'échec inclut une erreur réseau, un délai dépassé ou une réponse HTTP non réussie. Il ne suffit pas de tester uniquement si fetch() a levé une exception : fetch() ne rejette pas automatiquement les réponses HTTP 4xx/5xx.

Définissez une URL principale et une URL de secours dans la configuration du frontend. Implémentez une fonction de requête qui utilise AbortController avec un délai maximal, vérifie response.ok et bascule vers l'URL secondaire en cas d'échec.

Attention : la bascule ne peut pas garantir que le backend de secours fonctionnera. Les deux backends doivent être testés séparément et partager une configuration compatible, notamment la même base de test si l'application doit afficher les mêmes données.

Tests obligatoires

• Test A : backend Railway disponible — la requête aboutit sur Railway.

• Test B : Railway indisponible ou URL volontairement erronée — la requête bascule vers Vercel.

• Test C : les deux backends indisponibles — l'interface affiche un message d'erreur compréhensible.

• Test D : la requête POST ne crée pas deux fois un même élément en cas de nouvelle tentative. Expliquez les précautions nécessaires pour éviter les doublons.

```
git add frontend README.md
git commit -m "feat: ajouter le basculement vers le backend de secours"
git push
```

Partie 9 — Validation finale et démonstration (5 points)

• Démontrer l'interface en production.

• Montrer les URL du frontend, du backend Vercel et du backend Railway.

• Montrer les commits et le dépôt GitHub.

• Montrer les variables configurées dans les tableaux de bord sans révéler leurs valeurs secrètes.

• Démontrer les tests de bascule et expliquer les limites d’une architecture à deux backends.

Barème récapitulatif

Partie	Points
1. Git et structure	10
2. Backend Node.js / Express	15
3. MongoDB Atlas / Mongoose	15
4. Frontend HTML/CSS/JS	10
5. GitHub et documentation	10
6. Frontend Vercel	10
7. Backends Vercel + Railway	15
8. Bascule automatique	10
9. Validation finale	5
TOTAL	**100**

Checklist à remettre

- ■ URL du dépôt GitHub
- ■ URL du frontend Vercel
- ■ URL du backend Vercel (/api/health)
- ■ URL du backend Railway (/api/health)
- ■ Historique des commits conforme
- ■ README complet et .env.example présent
- ■ Aucun secret dans GitHub
- ■ Démonstration de bascule réussie

Important : la règle 0.0.0.0/0 et le modèle de mot de passe imposé sont des exigences pédagogiques de l’exercice, pas des recommandations de sécurité pour un système réel. Après l’examen, supprimer l’accès réseau global et faire tourner les identifiants.